

CS259-L:Comparative Analysis of Graph Algorithm Acceleration Using CUDA and OpenCL

Sowmya Rao

Computer Science Department, San Jose State University

San Jose, USA, CA

dhaheshwarsowmya.rao@sjsu.edu

ABSTRACT

Graph algorithms are fundamental to social network analysis, search engine indexing, and routing protocols. However, irregular memory access patterns pose challenges for traditional CPUs. This paper evaluates Breadth-First Search (BFS), PageRank, and Single Source Shortest Path (SSSP) across CPU, NVIDIA CUDA, and OpenCL. Our results demonstrate that while CPUs excel at micro-scales, GPU platforms provide transformative speedups for massive datasets.

1 INTRODUCTION

The data-driven nature of graph algorithms often leads to throughput bottlenecks on CPUs, which are optimized for sequential logic. GPUs address this by using thousands of small, efficient cores to perform parallel operations. This study identifies the transition point where GPU acceleration overcomes the overhead of kernel launches and memory transfers.

2 ARCHITECTURAL COMPARISON

The three platforms—CPU, CUDA, and OpenCL—differ significantly in their core design and primary goals. CPUs are built around a small number of powerful cores optimized for general-purpose logic and sequential processing. In contrast, both CUDA and OpenCL leverage thousands of smaller, more efficient cores designed for parallel workloads. Where they diverge is in scope: CUDA is a proprietary NVIDIA framework purpose-built for high-performance computing on NVIDIA hardware, while OpenCL is a cross-platform standard designed to run across a variety of devices and vendors.

3 ALGORITHM IMPLEMENTATIONS

3.1 Breadth-First Search (BFS)

BFS explores graph structures layer by layer, systematically visiting all neighbors of a node before moving to the next level. At its core, BFS maintains a frontier—the set of currently active nodes—and expands it outward until all reachable nodes have been visited. In parallel environments, the primary challenge is managing this frontier without race conditions, since multiple threads may attempt to visit the same node simultaneously. In the implementation:

- **CPU:** Uses a standard `std::queue` alongside a simple sequential check to mark visited nodes, making it

straightforward but limited to single-threaded execution.

- **CUDA:** Implements `atomicCAS` (Compare-And-Swap) to ensure only one thread wins the right to visit a node and add it to the frontier, while `atomicAdd` efficiently accumulates results across threads without conflict.
- **OpenCL:** Uses `atomic_cmpxchg` for visitation logic, `atomic_inc` to safely claim a unique index in the output buffer, and `atomic_add` to perform thread-safe aggregation of shared data.

3.1.1 Other Implementation

This implementation could have load balancing issues—where uneven node degrees cause some threads to do significantly more work than others, leaving remaining threads idle and wasting computational resources—which [1] addresses through Virtual-BFS (VBFS), a technique that partitions high-degree nodes into “virtual vertices” of uniform size K , ensuring balanced workloads across the SIMT architecture and reducing warp divergence.

3.2 PageRank

PageRank is an iterative algorithm that assigns importance scores to nodes based on the quality and quantity of their incoming links, with the core idea that a node is important if other important nodes point to it. At each iteration, every node redistributes its current rank to its neighbors, and this process repeats until the scores converge.

$$PR(u) = \frac{1-d}{N} + d \times \sum_{v \in B_u} \frac{PR(v)}{L(v)} \quad (1)$$

where d is the damping factor (typically 0.85), N is the total number of nodes, B_u is the set of nodes linking to u , and $L(v)$ is the out-degree of v . The damping factor models the probability that a random web surfer continues clicking links rather than jumping to a random page.

A major challenge is synchronization; at the end of each iteration, every platform must check whether the PageRank scores have converged—that is, whether the change in scores between iterations has fallen below a threshold small enough to stop. On the CPU, this check is straightforward since all computation happens in the same memory space, allowing the convergence condition to be evaluated directly after each iteration with no data transfer overhead. On CUDA, convergence is checked by copying the updated scores back from GPU memory to CPU memory via `cudaMemcpy`, after which the CPU evaluates the delta and signals whether another

iteration is needed. OpenCL handles this similarly but requires the explicit `CL_TRUE` flag during buffer reads to ensure the transfer is fully completed and the data is ready before the CPU inspects it—without this flag, the read may return before the GPU has finished writing, leading to incorrect convergence checks.

3.2.1 Other Implementation

This implementation handles only static graphs, whereas [2] extends this to dynamic graphs by recomputing PageRank only for nodes affected by newly added or removed vertices, rather than processing the full graph. It further optimizes this by pruning vertices whose rank change falls below a tolerance threshold, ensuring that only the most impactful nodes are recomputed at each iteration.

3.3 Single Source Shortest Path (SSSP)

Implemented via Bellman-Ford, SSSP is computationally intensive due to repeated edge relaxations—a process where every edge in the graph is checked to determine whether a shorter path to a destination node can be found, and this is repeated for up to $N-1$ iterations where N is the number of nodes. On the CPU, this is handled sequentially, making it straightforward but slow for large graphs as every edge must be processed one at a time. On CUDA, parallel edge relaxations are performed across thousands of threads simultaneously, with `atomicMin()` ensuring that when multiple threads attempt to update the shortest distance to the same node, only the minimum value is safely written without data corruption. OpenCL faces the same concurrency challenge but lacks a native floating-point atomic minimum operation, requiring a manually written Compare-And-Swap (CAS) loop that repeatedly reads the current value, computes the minimum, and attempts to write it back until the operation succeeds without interference from another thread.

3.3.1 Other Implementations

This implementation differs from the I-BF approach proposed in [3] in how it handles load balancing. In this code, assigning one thread per edge naturally avoids load imbalance since every thread handles exactly one relaxation regardless of how many edges a node has. In contrast, I-BF assigns one thread per node, meaning a high-degree node with 1,000 edges forces its single thread to perform 1,000 relaxations while threads assigned to low-degree nodes finish almost instantly and sit idle. To fix this, I-BF introduces virtual warp, which assigns an entire warp of 32 threads to each node instead of a single thread, so the 1,000 edges of a high-degree node are distributed across all 32 threads. For even larger imbalances at the block level, the dynamic algorithm further splits a node’s edge list into equal-sized chunks and distributes each chunk to a separate thread block, so no single block is overloaded. Together, virtual warp and the dynamic algorithm ensure that both threads within a warp and blocks across the GPU share edge relaxations evenly, which is the core load balancing contribution of I-BF that this simpler one-thread-per-edge implementation does not

need since its thread assignment already guarantees uniform work distribution by design.

4 PERFORMANCE RESULTS

The code has been tested on CPU and NVIDIA A100 GPU for CUDA and OpenCL implementations.

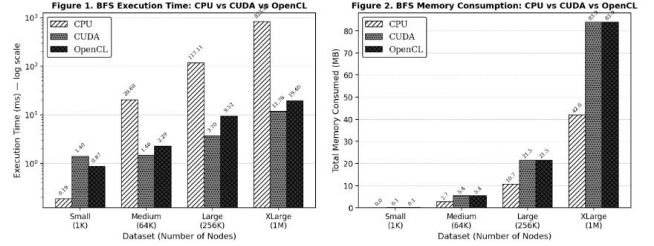


Figure 1: BFS Execution Time and Memory Consumption: CPU vs CUDA vs OpenCL

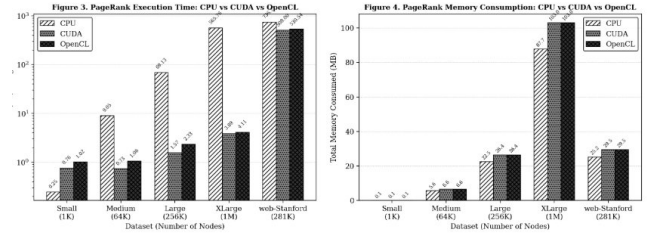


Figure 2: PageRank Execution Time and Memory Consumption: CPU vs CUDA vs OpenCL

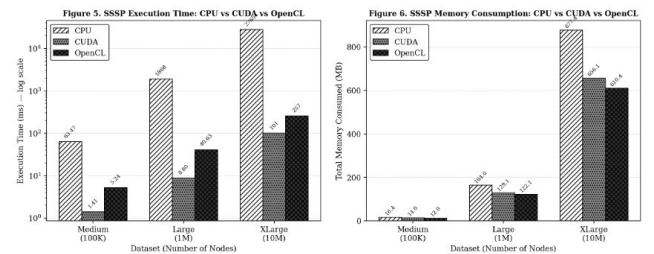


Figure 3: SSSP Execution Time and Memory Consumption: CPU vs CUDA vs OpenCL

5 SUMMARY/CONCLUSION

This paper evaluated three parallel graph algorithms—BFS, PageRank, and SSSP—across CPU, CUDA, and OpenCL platforms, demonstrating that while CPUs remain more efficient at small scales due to negligible memory transfer

overhead, GPUs provide substantial performance gains as graph size grows, with CUDA consistently outperforming OpenCL owing to its optimized NVIDIA-native toolchain. However, the PageRank implementation did not perform as expected on the real-world web-Stanford dataset from the SNAP collection, as the absence of load balancing caused significant inefficiencies when handling the highly irregular structure typical of real-world graphs, where node degrees vary drastically and cause uneven work distribution across threads. Future work should explore dynamic scheduling, load-aware frontier management, and hybrid CPU-GPU approaches to address these irregularities and further close the performance gap at intermediate scales.

REFERENCES

- [1] H. Wen and W. Zhang, “Improving Parallelism of Breadth First Search (BFS) Algorithm for Accelerated Performance on GPUs,” in Proc. IEEE High Performance Extreme Computing Conf. (HPEC), Waltham, MA, USA, Sep. 2019, pp. 1–7.
- [2] S. Sahu, K. Kothapalli, and D. S. Banerjee, “GPU-Accelerated Dynamic Frontier PageRank,” in Proc. Supercomputing India (SCI), Bangalore, India, 2025, pp. 1–8.
- [3] M. Nazarifard and D. Bahrepour, “Efficient Implementation of the Bellman-Ford Algorithm on GPU,” in Proc. IEEE 4th Int. Conf. Knowledge-Based Engineering and Innovation (KBEL), Tehran, Iran, Dec. 2017, pp. 773–778.